

Especificação do Sudoku

Problema escolhido: Resolvedor de Sudoku

Gabriel

TAG OBRIGATÓRIA: [P4-ETAPA-01]

1. Descrição do problema

O Sudoku é um quebra-cabeça de lógica jogado em um tabuleiro quadrado dividido em regiões menores, no qual algumas células já vêm preenchidas com números e as demais estão vazias. O objetivo é completar as células vazias respeitando um conjunto fixo de regras de unicidade.

O contexto de aplicação é o de uma ferramenta de apoio à resolução de quebra-cabeças: dado um tabuleiro parcialmente preenchido, o sistema deve determinar se existe uma solução válida e, em caso afirmativo, apresentá-la. Esse tipo de problema é amplamente utilizado como estudo de caso em disciplinas de programação porque envolve manipulação de estruturas de dados, verificação de restrições e busca em espaço de soluções — características que aparecem, com “sotaques” diferentes, em cada um dos quatro paradigmas de programação.

O problema será tratado na sua forma clássica: tabuleiro 9x9, dividido em 9 regiões (blocos) de 3x3, com dígitos de 1 a 9.

2. Objetivo

O sistema deverá ser capaz de:

- Receber um tabuleiro de Sudoku 9x9 parcialmente preenchido.
- Verificar se o tabuleiro fornecido é válido (não fere nenhuma regra do jogo no estado inicial).
- Encontrar uma solução completa para o tabuleiro, quando ela existir.
- Informar quando o tabuleiro não possui solução.
- Verificar se uma solução completa fornecida pelo usuário está correta.

3. Entradas

- Um tabuleiro 9x9, representado por uma matriz (ou estrutura equivalente) de inteiros de 0 a 9, onde:
 - Valores de 1 a 9 representam dígitos já preenchidos.
 - O valor 0 (ou um marcador equivalente, como espaço em branco) representa uma célula vazia.
- Opcionalmente, para a função de verificação de solução: um segundo tabuleiro 9x9 totalmente preenchido, a ser validado contra o tabuleiro original.

4. Saídas

- Para a operação de resolução:
 - O tabuleiro 9x9 completo, com todas as células preenchidas, caso exista solução.
 - Uma indicação explícita de que o tabuleiro não possui solução, caso não exista nenhuma configuração válida.
- Para a operação de validação de tabuleiro inicial:
 - Um indicador booleano (válido/inválido) e, preferencialmente, a lista de conflitos encontrados (célula e regra violada).
- Para a operação de verificação de solução fornecida:
 - Um indicador booleano (correta/incorreta) e, em caso de incorreção, a lista de células que violam alguma regra.

5. Regras do problema

1. O tabuleiro possui 9 linhas, 9 colunas e 9 blocos de 3x3, totalizando 81 células.
2. Cada célula deve conter um número inteiro entre 1 e 9 (não são permitidos zeros, letras ou valores fora desse intervalo na solução final).
3. Cada linha deve conter os números de 1 a 9 sem repetição.
4. Cada coluna deve conter os números de 1 a 9 sem repetição.
5. Cada um dos 9 blocos 3x3 deve conter os números de 1 a 9 sem repetição.
6. Os números já preenchidos no tabuleiro inicial são fixos e não podem ser alterados pela solução.
7. Um tabuleiro é considerado “resolvido” quando as 81 células estão preenchidas e as regras 3, 4 e 5 são satisfeitas simultaneamente.
8. Um tabuleiro pode não ter solução, ter exatamente uma solução, ou ter múltiplas soluções; para os fins deste trabalho, o

sistema deve encontrar **uma** solução válida quando existir ao menos uma.

6. Casos de exemplo

Exemplo 1 — Tabuleiro quase completo (uma célula vazia) Entrada: tabuleiro válido com todas as células preenchidas exceto a célula (linha 5, coluna 5), que está vazia, e cujo valor correto é 5. Saída esperada: tabuleiro completo com o valor 5 na célula (5,5).

Exemplo 2 — Tabuleiro vazio Entrada: tabuleiro com as 81 células vazias (nenhuma restrição inicial). Saída esperada: qualquer tabuleiro completo que satisfaça as regras 3, 4 e 5 (existem múltiplas soluções possíveis; qualquer uma é aceita).

Exemplo 3 — Tabuleiro parcialmente preenchido com solução única Entrada: um Sudoku clássico de dificuldade “fácil”, com cerca de 35 a 40 células preenchidas. Saída esperada: o tabuleiro completo correspondente à sua única solução.

Exemplo 4 — Tabuleiro sem solução (conflito direto) Entrada: tabuleiro em que a mesma linha contém o número 5 em duas células diferentes. Saída esperada: mensagem indicando que o tabuleiro é inválido, sem tentativa de resolução.

Exemplo 5 — Tabuleiro sem solução (conflito indireto) Entrada: tabuleiro sem repetições diretas em linhas, colunas ou blocos, mas cujas restrições combinadas tornam impossível preencher uma ou mais células (por exemplo, uma célula vazia cuja linha, coluna e bloco já contém, juntos, todos os valores de 1 a 9). Saída esperada: mensagem indicando que o tabuleiro não possui solução.

Exemplo 6 — Verificação de solução fornecida Entrada: o tabuleiro original e uma proposta de solução completa em que uma célula foi preenchida incorretamente (repete um valor já existente na mesma coluna). Saída esperada: indicação de que a solução é inválida, apontando a célula e a coluna em conflito.

7. Casos-limite

1. **Tabuleiro já completo na entrada:** todas as 81 células já preenchidas. O sistema deve apenas validar se a solução é correta, sem necessidade de busca.
2. **Entrada malformada:** matriz com dimensões diferentes de 9x9, ou contendo valores fora do intervalo permitido (por exemplo, números negativos, maiores que 9, ou caracteres não numéricos). O sistema deve rejeitar a entrada com uma mensagem de erro clara, sem tentar processá-la como um tabuleiro válido.
3. **Tabuleiro inicial inválido:** o tabuleiro fornecido já contém, por si só, uma violação das regras (por exemplo, dois números iguais na mesma linha, coluna ou bloco), mesmo antes de qualquer tentativa de preenchimento. O sistema deve identificar essa situação antes de iniciar a busca por solução.
4. **Tabuleiro com múltiplas soluções:** quando mais de uma solução válida existir, o sistema deve retornar qualquer uma delas, deixando explícito (na documentação da implementação) que a unicidade da solução não é garantida nem verificada.

8. Restrições

- O sistema não tratará variantes do Sudoku (como Sudoku 4x4, 16x16, “X-Sudoku”, “Sudoku killer” ou versões com regiões irregulares). Apenas a variante clássica 9x9 com blocos 3x3 será considerada.
- O sistema não terá interface gráfica; a entrada e a saída serão tratadas em formato de dados simples (matriz/lista), cabendo a cada implementação de paradigma decidir a forma concreta de representação textual.
- O sistema não é responsável por gerar novos quebra-cabeças de Sudoku (geração de tabuleiros iniciais), apenas por resolver e validar tabuleiros fornecidos.
- Não é objetivo do sistema avaliar a “qualidade” ou o nível de dificuldade de um tabuleiro.
- Aspectos de desempenho (tempo de execução em tabuleiros muito difíceis) não serão tratados como requisito formal nesta etapa, apenas correção do resultado.

9. Principais conceitos do domínio

- **Tabuleiro:** a grade 9x9 completa que representa o estado do jogo.
- **Célula:** uma posição individual do tabuleiro, identificada por linha e coluna, que pode estar vazia ou conter um dígito de 1 a 9.
- **Linha, Coluna e Bloco:** os três tipos de “grupo” de 9 células sobre os quais a regra de não repetição é aplicada.
- **Valor fixo (pista):** um número presente no tabuleiro inicial, que não pode ser alterado durante a resolução.
- **Candidato:** um valor que, em um dado momento da resolução, ainda pode ser colocado em uma célula vazia sem violar as regras conhecidas até então.
- **Solução:** um tabuleiro completo (sem células vazias) que satisfaz todas as regras do problema.
- **Conflito:** uma situação em que um valor colocado em uma célula viola a regra de não repetição em relação à sua linha, coluna ou bloco.

10. Adequação aos quatro paradigmas

- **Imperativo:** o problema se resolve naturalmente por meio de laços aninhados percorrendo linhas, colunas e blocos, com estruturas de controle explícitas (condicionais e repetição) para verificar restrições e implementar o algoritmo de backtracking

(tentativa e retrocesso) célula a célula.

- **Orientado a objetos:** os conceitos do domínio mapeiam-se diretamente em classes — Tabuleiro, Célula, e possivelmente Regra ou Validador — permitindo encapsular o estado do tabuleiro e expor métodos como `ehValido()`, `resolver()` e `obterCandidatos(linha, coluna)`.
- **Funcional:** a resolução pode ser expressa como uma função pura que recebe um tabuleiro (imutável) e retorna um novo tabuleiro ou Nada/Nothing, usando recursão para explorar o espaço de soluções e funções de alta ordem (`map`, `filter`) para calcular candidatos e verificar restrições, sem uso de estado mutável.
- **Lógico:** o Sudoku é um exemplo canônico de problema de satisfação de restrições, adequando-se diretamente à programação em lógica — cada regra do jogo (linha, coluna, bloco sem repetição) pode ser expressa como uma cláusula, e o motor de inferência (por exemplo, o mecanismo de unificação e backtracking do Prolog) encontra a solução automaticamente a partir da declaração das restrições, sem que seja necessário descrever o algoritmo de busca explicitamente.

Dessa forma, o mesmo problema pode ser modelado de quatro formas conceitualmente distintas — como sequência de instruções, como objetos colaborando, como funções compostas e como um conjunto de fatos e regras — sem que sua especificação dependa de nenhuma linguagem específica.

11. Linguagens inicialmente consideradas

Paradigma	Linguagem considerada	Justificativa
Imperativo	C	Linguagem imperativa clássica, com controle explícito de memória e estruturas de repetição, adequada para implementar o backtracking manualmente sobre uma matriz.
Orientado a objetos	Java	Suporte maduro a classes, encapsulamento e coleções, permitindo modelar Tabuleiro e Célula como objetos com responsabilidades bem definidas.
Funcional	Haskell (alternativa: F# ou Elixir)	Linguagem funcional pura, com imutabilidade e recursão como mecanismos centrais, favorecendo uma implementação declarativa da busca por solução.
Lógico	Prolog	Linguagem lógica de referência, na qual restrições do tipo “sem repetição em linha/coluna/bloco” podem ser expressas diretamente como cláusulas, delegando a busca ao motor de inferência.